Perfect Consign v2 — Hostile Review, Round 6 (HEAD `8c913c1`)

Supersedes `HOSTILE-REVIEW-f7516ba.md`. Every earlier finding is carried forward with a status. New findings continue the numbering at N31.

Scope: 2 commits since `f7516ba` (`93e2eb1` "Add the /admin platform portal and land the pending store work", `8c913c1` "Align the platform admin UI with the store app chrome"). That is **157 files, +10,159/−261**, including **18 new migrations (019–036, +3,454 lines of SQL)** and 39 changed test files. This is by a wide margin the largest diff I have reviewed on this repository: it lands the offline register, the commission-rule engine, the markdown queue, the consignor portal, booth leases, vendor letters, the platform admin portal, and the fixes for N25–N30 all at once.

The server was re-provisioned from scratch: the `consign` database was dropped and recreated, every `stall_*` and `poster_*` role dropped, then `provision-platform`, `enable-stall-app`, `provision-tenant mall`, `migrate platform`, `migrate tenant-all`, `migrate platform-after` and `set-platform-admin` were run. PHP 8.5.12, PostgreSQL 18.6 on port 5436, Composer 2.10.3, xdebug.

Not run, by standing instruction: mutation testing (N2 is carried as not re-tested).

Verdict

The three HIGHs from Round 5 are genuinely closed, and closed the right way. But HEAD ships with `composer check` red in three places, and the new offline register trusts the browser to tell the ledger what the commission and tax on a sale were.

First, credit where it is earned. N25, N26 and N27 are gone and I confirmed each by execution rather than by reading. `platform.job_mail_sent` and `platform.override_sign_in_failures` no longer exist on the platform schema at all; the data moved into each tenant's schema (`029`, `033`) and the platform copies were dropped (`032`, `034`) by migrations that refuse to run until every tenant has the local table. The override lockout now goes through three `SECURITY DEFINER` functions and the poster role has `EXECUTE` on those and nothing on the table — I tried a direct `DELETE` as `stall_poster` and it was refused. `SaleCeiling` no longer touches `platform.tenants`. The enqueue path is gated through `platform.enqueue_vendor_letter()`, which derives the tenant from the session role instead of trusting an argument, and a direct `INSERT INTO platform.jobs` as a poster is now `permission denied`. That is exactly the shape I asked for in Round 5, and it was done in a way that the `prove:tenant` suite will catch if it regresses.

Now the problems.

HEAD does not pass its own gate. `composer check` is the contract this repository makes with itself, and at `8c913c1` three of its twelve steps fail. PHPMD reports four violations (two in `RegisterFeatures::register()`, one each in `AdminApp::show` and `AdminAuditController::index`). The unit suite has one error and one failure: `LetterDeskTest` calls the new enqueue gate as the superuser and is correctly refused by the very guard that fixed N25, and `MigrationBoundaryTest` still counts `MailboxSeal` statements by matching `left(gmail_app_password`, a string the N28 fix removed. The HTTP suite failed once in the full run (`FloorCoverageTest`, "vendor vfde810 was not on the page") and passed when I ran it alone, which makes it flaky rather than broken — but flaky in a suite that takes five minutes is how a team learns to ignore red. The prior five rounds all had a green `composer check` at HEAD; this is the first that does not, and it arrived with the largest commit. That is N31.

The offline register trusts the client. The design in `027_offline_snaps.sql` is clever: when the network drops mid-payment, the browser stores each line in `localStorage` with a `posting_key`, and on reconnect posts it to `/pos/snap` and then `/pos/sync`. The sync function is idempotent on `posting_key` — I replayed the same key and got one journal, which is exactly right, and C5's lesson has clearly been learned. But the snap row carries `price_minor`, `tax_minor`, `commission_minor`, `commission_bps` and `tax_bps` **as supplied by the browser**, and `stall_sync_snap()` posts those numbers straight into `journal_lines` and `sale_lines`. A `BEFORE INSERT` trigger then temporarily rewrites `items.commission_bps` and `mall.tax_bps` so that the normal commission trigger agrees with the snapped figures. I inserted a snap as `stall_poster` for an item whose vendor is on 20% commission, with the mall at 8.75% sales tax, and asserted `commission_minor = 0, tax_minor = 0, commission_bps = 0, tax_bps = 0`. It posted. Journal 218: `Cash 50.00 / Vendor payable 50.00`, no commission line, no tax line. The store's own commission engine (`stall_commission_for`) says 2000 bps for that item on that date; the ledger says zero, because the browser said so. A cashier with the dev tools open — or simply a stale tab from before a rate change — can post a sale at any commission and any tax rate. That is N32, and it is HIGH because it is a money-rule bypass reachable from the till.

The same table also has `GRANT ALL … TO stall_poster`. I updated a posted snap's `price_minor` to 1 and then deleted the row, as the poster, after the journal had been written. The journal is still correct — append-only held — but the record that explains why that journal has those numbers is gone, and `stall_apply_snap_price()`, which uses the snap row to refuse a re-priced sale, no longer has anything to match against. That is N33.

The consignor portal does not use the role built for it. `025_vendor_role.sql` creates `stall_vendor` — `NOINHERIT NOLOGIN`, granted only `SELECT` on five `security_barrier` views keyed on `current_setting('stall.vendor_id')`. That is the right design. But `grep stall_vendor src/` returns nothing; the only member of `stall_vendor` is the superuser. `VendorPortal::assume()` does `SET ROLE` to the **poster** role, then `set_config('stall.vendor_id', …)`, then reads the views. The poster role has `SELECT` on the same views and `SELECT` on the underlying tables. So the isolation guarantee is "the PHP code sets the GUC to the right vendor", not "the database cannot show anyone else's rows". I confirmed as `stall_poster` that setting the GUC to a different vendor id returns that vendor's items. Nothing in the portal is wrong today; the code sets the GUC correctly. But the whole point of building `stall_vendor` was to make the correctness not depend on the PHP. That is N34.

Password-reset links can be poisoned through the `Host` header. `requestOrigin()` in `public/index.php` builds the base URL for reset and consignor-entry mail from the raw `Host` header, with no canonical-host setting anywhere in the configuration. I requested an admin password reset with `Host: evil.example` and the mail that went out read `http://evil.example/admin/reset/9kPC1aqe…`. The same works via `X-Forwarded-Host`. The classic attack is to trigger a reset for the victim's address from the attacker's machine; the victim receives a genuine-looking email from the real sender, clicks, and hands the token to the attacker's host. The owner reset and consignor magic-link paths use the same `origin`. Behind a correctly configured reverse proxy that rewrites `Host` this is mitigated by deployment, but the application should not depend on that and nothing in the repo documents it. That is N35, HIGH.

The admin portal is good, with one structural gap. `/admin` is well put together: argon2 hashes, per-address lockout with `client_address`, append-only `admin_audit` with event-trigger guards (I confirmed `stall_app` cannot `UPDATE` or `DELETE` it), `credentials_version` checked on every request, CSRF on every `POST`, a slug-confirmation gate on wipe, and `platform.wipe_tenant()` refusing any role but `stall_app`. The gap is the session model. "Enter as" opens a tenant session in the **same** PHP session as the admin session and then restores the admin keys alongside it. I confirmed over HTTP: after entering the mall as owner, `/mall/dashboard` and `/admin/tenants` both answer 200 on one cookie. Logging out of the tenant leaves the admin signed in; logging out of the admin leaves the tenant signed in. And `requireAdmin()` writes `platform_last_seen_at` but never reads it — there is no idle or absolute expiry on an admin session, where the tenant session has both (12 h idle, 24 h absolute). That is N36.

And C3 has changed shape rather than closed. The one-click wipe that used to be a `reset-test-data` route on the demo is now a first-class `/admin` feature that drops a production tenant's schema, poster role and owner rows with no backup step, no soft-delete window and no two-person rule. The confirmation is typing the slug. That is a stronger control than before, and the audit row is written first, so I am moving C3 from OPEN to PART — but a buyer's due-diligence team will ask "what happens when an admin account is phished", and today the answer is "every tenant can be deleted in under a minute with no recovery". That is worth saying in the C3 row and in the vision document.

What is still wrong, in order of importance:

1. **N32 (new, HIGH): the offline snap posts whatever commission and tax the browser supplies.** Confirmed by execution: a 50.00 sale of a 20%-commission item posted with zero commission and zero tax.
2. **N35 (new, HIGH): reset and magic-link mail is built from the `Host` header.** Confirmed by execution: `http://evil.example/admin/reset/<token>` delivered.
3. **N31 (new, HIGH as process): `composer check` is red at HEAD** — PHPMD (4), unit (1E+1F), HTTP (1 flaky). Coverage dipped below the 90% floor (89.77%) for the first time.
4. **N34 (new, MEDIUM): the consignor portal runs as the poster role; `stall_vendor` exists and is unused.**
5. **N36 (new, MEDIUM): admin and tenant sessions share one cookie with asymmetric logout, and the admin session never expires.**
6. **N33 (new, MEDIUM): `register_snaps` is fully writable by the poster after posting.**

What is now closed: N25, N26, N27 (all confirmed by execution), N28 (selector widened), N22's retention half (inside `stall_override_record`). N7 moves from "nominal" to genuinely resolved because N27 is gone. C5 moves to PART: the snap path has an idempotency key; store-use still does not.

Gate results at `8c913c1`

Gate	Result	Detail
`check:freeze`	**pass**	No stamped migration edited.
`check:max-lines`	**pass**	Nothing over 500.
`check:phpmd`	**FAIL (rc=2)**	`RegisterFeatures::register()` cyclomatic 10 / NPath 208; `AdminApp::show` boolean flag `$nav`; `AdminAuditController::index()` cyclomatic 10.
`phpstan` (L10)	**pass**	Clean.
`test` (unit)	**FAIL (rc=2)**	793 tests, 3,280 assertions, **1 error, 1 failure**. `LetterDeskTest::testOneConsignorIsFiledAndListed` → `42501`

		`a poster role must enqueue the letter`. `MigrationBoundaryTest::testAMallTenantGrantsThePosterRole` → `0 !== 1` seals.
`test:integration`	**pass**	33 tests, 257 assertions.
`test:http`	**FAIL (rc=2), flaky**	80 tests, 1,229 assertions, 1 error in the full run (`FloorCoverageTest::testADiscountAndATaxWaiverStayOnTheReceipt`, "vendor vfde810 was not on the page"). Passed in isolation (26 assertions). The vendor row existed in the database when I looked.
`prove:obligations`	**pass**	
`prove:race`	**pass**	3m11s.
`prove`	**pass**	64 `prove_*.sql` + `wipe-backup`, up from 57. New: `prove_offline_sync`, `prove_race_snap_a/b`, `prove_vendor_portal`, `prove_vendor_letter`, `prove_commission_rules`, markdown, leases.
`prove:tenant`	**pass**	
`check:worker`	**pass**	New gate; replaces the CI "Run one worker pass" step and additionally asserts no claimed job is left unfinished and nothing due remains. Better than what it replaced.
coverage	**89.77%** (20,973/23,363)	**Below the 90% floor** (`coverage-run.php` prints "line coverage 89.77% is below 90%"). Down from 91.13%. 18 files at 0.0%. 906 tests, 4,856 assertions.

What I executed, not read

- All twelve gates, plus coverage, on a fresh cluster.
- `psql` as `stall_poster`: grants matrix on `register_snaps`, `commission_rules`, `vendor_entry_tokens`, `job_mail_sent`, the five `stall_vendor_*` views and the override functions; direct `DELETE FROM admin_audit` as `stall_app` (refused); `platform.wipe_tenant()` as `stall_poster` (refused) and `stall_app` (permitted).
- `stall_sync_snap()` with client-asserted zero commission and zero tax → posted (N32). Replay of the same key → one journal (idempotent, good). Commission greater than price → refused by `journal_lines_amounts_chk` (good). `UPDATE` / `DELETE` of a posted snap row as poster → succeeded (N33).
- `set_config('stall.vendor_id', <other vendor>)` as `stall_poster` → other vendor's 8 items returned (N34).
- HTTP against a `php -S` instance: admin login (303 → `/admin/tenants`), 12 bad passwords → "Too many sign-in attempts" (lockout works), enter-as owner → 303 → `/mall/dashboard` → 200, then `/admin/tenants` → 200 on the same cookie; tenant logout → admin still 200; admin logout → `/mall/pos` still 200 (N36).
- `POST /admin/forgot` and `POST /mall/forgot` with `Host: evil.example` and with `X-Forwarded-Host: evil2.example` → reset link in the mail file uses the attacker host (N35).
- Response headers on `/admin/login`: only `Set-Cookie … HttpOnly; SameSite=Lax`. No `X-Frame-Options`, `Content-Security-Policy`, `X-Content-Type-Options` or `Strict-Transport-Security` anywhere in `src/` or `public/index.php` (L8).

Status ledger

Round 5's ledger had 62 rows. This one has 70: the 62 carried forward and eight new rows (N31–N37, L8). Tally: **45 resolved, 11 partial, 3 open, 2 withdrawn, 1 not re-tested, 8 new.**

ID	Finding	Status	Evidence at `ea1b5c4`
C1	Live Gmail app password committed	**RES (revoked)**	Owner-confirmed revocation; carried as closed. HEAD clean. Residual is L7.
C2	No upgrade path / deploy wipes	**PART**	Forward-only 002–036 now, with `PLATFORM_AFTER` ordering for the two drops. "create a new database" still appears in `TenantMigrator`, `MigrationRecord`, `SchemaMigrator`; README still says an existing database is not upgraded. The migration *machinery* is now mature enough that this sentence is simply out of date.
C3	One-click wipe on production demo	**PART (changed shape)**	`reset-test-data` still exists for dev. The wipe is now also a first-class `/admin` feature: slug confirmation, CSRF, audit-before-act, `stall_app`-only `SECURITY DEFINER`. Still no backup step, no soft-delete window, no second approver. See N36 for why a phished admin is the realistic path.
C4	`wipe-tenant.php` drops platform FKs	**PART**	`wipe-tenant.php` now prefers `platform.wipe_tenant()` and only falls into the legacy FK-dropping path when the function is absent. Legacy path still present.
C5	Store use has no idempotency key	**PART**	The new snap path has one (`register_snaps.posting_key UNIQUE`, replay returns the same journal — proved). `StoreUse` still posts without one.
H1	Worker runs as DB superuser	**RES**	Unchanged; `check:worker` now runs as `stall_worker` in CI.
H2	Failed job finished, never retried	**RES**	Unchanged.
H3	`enqueue()` check-then-insert race	**RES**	Unchanged.
H4	Late fees bill all history	**RES**	Unchanged.
H5	Demo deploys without waiting for `check`	**RES**	Unchanged — but see N31: `check` itself is red.

H6	(withdrawn earlier)	WD	—
H7	Migration checksum rewrites itself	**RES**	Unchanged.
H8	Audit trail is decorative	**PART**	34 tenant audit call sites. The admin portal audits well (`admin_audit`, append-only, event-trigger guarded). Tenant side still lacks period close, adjustments, manual journals, POS, settings, password change. `request_id` still from the constructor.
H9	`Permission` used in one place	**PART**	Three call sites. Unchanged.
M1	Tests that cannot fail	**RES**	Unchanged.
M2	SMTP header CRLF injection	**RES**	Unchanged.
M3	Reset mail failure swallowed	**RES**	Unchanged.
M4	Owner reset falls back to company inbox	**RES**	Unchanged.
M5	Gmail app password plaintext	**RES**	Unchanged. N28 caveat now closed.
M6	CSV unescaped / formula injection	**RES**	Unchanged.
M7	`.cursor/rules/` shipped	**PART**	4 files still in the product repo.
M8	`overdue()` inverted	**RES**	Unchanged.
M9	Web tier cannot enqueue	**RES**	Reaffirmed: web enqueues only through `platform.enqueue_vendor_letter()`, direct `INSERT` refused.
M10	Reconcile proves the view, not the report	**RES (read only)**	Unchanged.
M11	Late-fee journals dated "today"	**RES**	Unchanged.
L1	`sendJson()` duplicated	**RES**	Unchanged.
L2	No `Cache-Control: no-store` on JSON	**RES**	Unchanged.
L3	`StoreUse::itemId()` null on empty code	**RES**	Unchanged.
L4	Real-looking identities in `dev_tenant.sql`	**RES**	Unchanged.
L5	`'mall-demo'` fallback password	**RES**	Unchanged.
L6	README is Windows-oriented	**RES**	Unchanged.
L7	No secret scanning in CI	**OPEN**	No `gitleaks` / `trufflehog` step in `check.yml`.
N1	Files at 0% coverage	**PART (regressed)**	**89.77%**, down from 91.13% and below the repo's own 90% floor. 18 files at 0.0%: `ReportsController`, `MissedSale`, `NsfController`, `VendorMemo`, `FloorController`, `MissedSaleController`, `VendorMemoController`, **`LetterController`** (new), **`letters.php`** (new), `MallSaleController`, `StoreUseController`, `LedgerReport`, `ReportCsvController`, `SavedReport`, **`Admin/reset.php`** (new), `vendor-memo.php`, `ReceiptMailer`, `missed-sale.php`. Thin: `LetterDesk` 34.6%, `BoothExtraController` 7.2%, `StoreUse` 6.0%, `AdminPasswordController` 14.8%, `ReceiptMailSettings` 20.0%, `JobSchedule` 32.2%, `JobQueue` 32.6%.
N2	Parallel mutation shards collide	**NR**	Mutation not run.
N3	`prove` red	**RES**	64 of 64.
N4	NSF actor client-supplied	**RES**	Unchanged.
N5	New features ship with no tests	**PART**	Much better this round: `prove_offline_sync`, `prove_race_snap_a/b`, `prove_vendor_portal`, `prove_commission_rules`, `AdminPortalTest` (HTTP, end-to-end incl. wipe), `LedgerPathTest` covers snap/sync over HTTP, `TenantBoundaryTest` now asserts the N25/N26 cross-tenant refusals. Still nothing drives `LetterController`, `letters.php`, or the admin reset page.
N6	`email_bounced` sticky	**RES**	Unchanged.
N7	Override has no throttling	**RES**	Was "nominal" because of N27. N27 is closed; the lockout now runs under the poster role via `stall_override_*`. Genuinely resolved.
N8	Mall sales: no audit, overlap, or cap	**RES**	Unchanged.
N9	Booth deposit: no drawer link, no audit	**RES**	Unchanged.
N10	Sale notice shows minor units	**RES**	Unchanged.
N11	`/reports*` prefix match	**RES**	Unchanged.
N12	`keep_days` has no CHECK	**WD**	—
N13	Job worker cannot run	**RES**	`check:worker` green.
N14	CSV guard corrupts negatives	**RES**	Unchanged.
N15	Booth deposits drop the booth	**RES**	Unchanged.
N16	Deploy script's fallback path	**RES (read only)**	`deploy-demo.sh` now: ensure-mail-key → migrate platform → tenant mall → tenant-all → platform-after. Still exits on failure.

N17	Reset-letter failure leaks username existence	**RES**	Unchanged.
N18	Override lockout shares the owner table	**RES**	Now a tenant-schema table behind `SECURITY DEFINER` functions (033).
N19	Mail retries resend to everyone	**RES**	Unchanged; `job_mail_sent` now tenant-local (029).
N20	Sealed-secret rollout	**RES**	Unchanged.
N21	Changed code is mostly never executed by a test	**PART**	`check:worker` is a real gate now. `SaleCeiling` is driven by `TenantBoundaryTest`. But `JobQueue` 32.6%, `JobSchedule` 32.2%, `LetterDesk` 34.6%, and the overall number fell below the floor.
N22	Override lockout per username, not per address	**PART**	Retention delete now inside `stall_override_record()` and runs under the poster (fixed). Per-address half not done; `override_sign_in_failures` has no `client_address` column where `admin_sign_in_failures` (036) does — the newer table got it right.
N23	Digests use the server mailbox; dead letters raise no alert	**OPEN**	Unchanged.
N24	Worker blast radius	**RES (accepted)**	Unchanged.
N25	`platform.job_mail_sent` cross-tenant	**RES**	**Confirmed by execution.** `platform.job_mail_sent` does not exist. Tenant-local `job_mail_sent` is `SELECT, INSERT` to the poster only. `TenantBoundaryTest` inserts into another schema's table and asserts the refusal.
N26	`platform.override_sign_in_failures` cross-tenant delete	**RES**	**Confirmed by execution.** Platform table dropped (034). Tenant-local table has `REVOKE ALL FROM PUBLIC`; poster has `EXECUTE` on `stall_override_attempts/record/clear` only. Direct `DELETE` as poster → `permission denied`.
N27	Override path throws under the tenant role	**RES**	**Confirmed by execution.** `SaleCeiling` calls `SELECT stall_override_attempts(?)` etc. No `platform.tenants` subquery remains. `TenantBoundaryTest` exercises lock and refusal as the poster.
N28	`MailboxSeal` malformed selector	**RES**	Selector is now `NOT (gmail_app_password ~ '^v1:[A-Za-z0-9+/]+=*$' AND char_length BETWEEN 20 AND 200)`. But the unit test still greps for the old `left(` text — that is one half of N31.
N29	`job_mail_sent` has no integration test and no retention	**PART**	Integration test exists now (`TenantBoundaryTest`). Still no retention: the table grows one row per recipient per letter per day, forever.
N30	Changed email gets a second letter	**RES (documented)**	Comment in `029` accepts it as intended behaviour. Fine.
N31	**`composer check` is red at HEAD; coverage below floor**	**NEW — HIGH (process)**	See below.
N32	**Offline snap posts browser-supplied commission and tax**	**NEW — HIGH**	Confirmed by execution. See below.
N33	**`register_snaps` fully writable by the poster after posting**	**NEW — MEDIUM**	Confirmed: `UPDATE 1`, `DELETE 1` on a posted row.
N34	**Consignor portal runs as the poster role; `stall_vendor` unused**	**NEW — MEDIUM**	Confirmed: only `stall` is a member of `stall_vendor`; GUC switch as poster reads any vendor.
N35	**Reset and magic-link URLs built from the `Host` header**	**NEW — HIGH**	Confirmed: `http://evil.example/admin/reset/…` delivered.
N36	**Admin and tenant sessions share a cookie; admin session never expires**	**NEW — MEDIUM**	Confirmed over HTTP.
N37	**Commission/tax triggers mutate `items` and `mall` rows mid-insert**	**NEW — LOW (design)**	Read, not raced. See below.
L8	**No security response headers**	**NEW — LOW**	Only `Set-Cookie` flags. No CSP, frame-ancestors, nosniff, HSTS.

HIGH

N31 — `composer check` is red at HEAD, and coverage is under the floor

`composer check` is twelve steps. At `8c913c1`, three fail and a fourth is below its own threshold.

PHPMD (rc=2). Four violations, all in files touched by this diff:

```
src/Bootstrap/RegisterFeatures.php:17   CyclomaticComplexity  register() = 10 (threshold 10)
src/Bootstrap/RegisterFeatures.php:17   NPathComplexity       register() = 208 (threshold 150)
src/Feature/Admin/AdminApp.php:112      BooleanArgumentFlag   show(..., bool $nav)
src/Feature/Admin/AdminAuditController.php:19  CyclomaticComplexity  index() = 10
```

Unit (rc=2). 793 tests, one error, one failure.

`LetterDeskTest::testOneConsignorIsFiledAndListed` → `SQLSTATE[42501] a poster role must enqueue the letter` from `enqueue_vendor_letter(date,uuid) line 8`. The test connects as the superuser and never `SET ROLE`s to a poster; `035_enqueue_session_role.sql` correctly refuses. The guard is right and the test is wrong. It needs to assume the poster role before calling `LetterDesk::file()`.

`MigrationBoundaryTest::testAMallTenantGrantsThePosterRole` → `Failed asserting that 0 is identical to 1` at line 179. The test counts "seal" statements by matching the text `left(gmail_app_password`. The N28 fix replaced that selector with a regex match, so the count is zero. The fix was right and the test is stale.

Both are five-minute fixes. The point is not the fixes; it is that they shipped. `H5` was resolved in Round 2 by making the demo deploy wait for `check`. If `check` is red, that protection is now blocking every deploy — or, worse, someone has started deploying around it.

HTTP (rc=2, flaky). `FloorCoverageTest::testADiscountAndATaxWaiverStayOnTheReceipt` threw `vendor vfde810 was not on the page` after creating the vendor and fetching `/parties`. The vendor row exists (I checked `webprobe`'s schema: `vfde810 | Ada vfde810 | active`). The same test passed alone. The `/parties` page renders every vendor with no paging or limit, so the likely cause is a request ordering race in the probe server rather than a product bug — but it needs a root cause, because an HTTP suite that fails one time in N is an HTTP suite nobody waits for.

Coverage. `coverage-run.php` itself prints `line coverage 89.77% is below 90%`. Coverage fell from 91.13% to 89.77% because 10,000 lines landed and the new controllers (`LetterController`, `letters.php`, `Admin/reset.php`) are at 0%.

Fix: make the two unit tests assume the poster role / match the new selector; split `RegisterFeatures::register()` and `AdminAuditController::index()`; replace the `$nav` flag with two methods or a value object; find the `FloorCoverageTest` race; add HTTP coverage for the letters page and the admin reset page. Then put `composer check` back to green and keep it there. Consider a branch-protection rule so a red `check` cannot merge to `main`.

N32 — the offline snap posts whatever commission and tax the browser supplies

`register-outbox.js` builds each queued line from `data-snaps` on the pay form, which includes `price`, `tax`, `commission`, `commission_bps` and `tax_bps` as computed by the server **at the time the page was rendered**. On reconnect it `POST`s those fields to `/pos/snap`. `SnapController::save()` validates ranges (`commission_bps ≤ 10000`, `tax_bps ≤ 100000`, quantity format) and inserts them into `register_snaps`. `stall_sync_snap()` then posts `snap.commission_minor` and `snap.tax_minor` directly into `journal_lines` and `sale_lines`, and `stall_apply_snap_price()` — a `BEFORE INSERT` trigger on `sale_lines` — temporarily overwrites `items.commission_bps` and `mall.tax_bps` with the snapped rates so the regular commission trigger (`sale_lines_00_rate`) short-circuits via `stall.snap_priced = '1'`.

Nothing recomputes commission or tax from the store's rules at sync time. The browser is the source of truth.

Executed as `stall_poster` in the mall tenant, against an item whose vendor is on 2000 bps and a mall at 8750 bps tax:

```
INSERT INTO register_snaps (posting_key, session_id, business_date, item_id,
    price_minor, tax_minor, commission_minor, commission_bps, tax_bps, quantity)
VALUES ('aaaaaaaa-…a001', <open session>, current_date, <item>, 5000, 0, 0, 0, 0, 1);
SELECT stall_sync_snap('aaaaaaaa-…a001');   --> 218

 line_no |      name      | debit | credit | subledger_ref
       1 | Cash           |  5000 |      0 |
       3 | Vendor payable |     0 |   5000 | 01a171aa-8368-…   (the vendor)

SELECT stall_commission_for(<item>, 5000, current_date);  --> 2000
```

Journal 218 pays the vendor 100% of a sale on which the store was owed 20%, and collects no sales tax on a taxable sale in a taxing jurisdiction. The sale line records `commission_minor = 0, tax_minor = 0`. The vendor's payable is overstated by 10.00; sales tax payable is understated by 4.38. Both figures will flow into the vendor's statement, the 1099 total, and the sales-tax report.

Three things make this worse than a theoretical dev-tools attack. First, the snap payload is captured when the pay modal renders, so a register that was open across a midnight rate change (a markdown ladder step, a commission-rule edit, a tax-rate update) will post the old figures and the ledger will not notice. Second, the "offline" path is triggered by *any* `fetch` failure on `/pos/pay` — a 502 from the proxy, a timeout, a CSRF expiry — not only a network outage, so it is reached more often than "the ISP went down". Third, the code that would have caught this (`stall_apply_commission_rate`) is deliberately disabled by the snap trigger.

The intent is clear and reasonable: an offline sale should post at the price the customer actually paid and the tax actually charged on the receipt. That is right for `price_minor` and arguably for `tax_minor` (the customer has already paid it). It is **not** right for commission. Commission is a contract between the store and the vendor, it is not visible on the receipt, and the customer's payment does not depend on it. The server knows the rule set and the business date; it should compute

commission at sync time from `stall_commission_for(item, price, business_date)` and ignore the client's figure. For tax, the honest approach is to compute from the rate in force on `business_date` and, if the client's figure disagrees, post the client figure (because that is what was collected) and write the variance to an "offline tax variance" line so it shows on the tax report rather than vanishing.

Fix: in `stall_sync_snap()`, compute `commission_minor := public.stall_commission_for(snap.item_id, snap.price_minor, snap.business_date) * …` and use that; drop `commission_bps` and `commission` from the snap payload entirely. For tax, compute the expected amount, post the collected amount, and record the delta. Remove the `items.commission_bps` / `mall.tax_bps` rewrite from `stall_apply_snap_price()` once commission is server-derived (see N37). Add a proof: `prove_offline_sync_rules.sql` that snaps with wrong client rates and asserts the journal matches the rule engine. One to two days.

N35 — password-reset and magic-link URLs are built from the `Host` header

`public/index.php::requestOrigin()` returns `scheme://` + the raw `Host` header. `AdminPasswordRecovery::request()`, the owner reset path and `VendorEntry::request()` all accept that `$origin` and concatenate the token onto it. `VendorEntry` validates the shape (`^https?://[A-Za-z0-9.-]+(:\d+)?$`) but not the value. There is no `STALL_PUBLIC_ORIGIN` / canonical-host setting anywhere in `src/`, `bin/`, `README.md` or `.env` handling.

Executed against a `php -S` instance:

```
POST /admin/forgot   Host: evil.example   email=admin@example.test   → 303
POST /mall/forgot    Host: evil.example   username=owner             → 303
POST /mall/forgot    X-Forwarded-Host: evil2.example                 → 303

mail file:
  Open this link within 30 minutes:
  http://evil.example/admin/reset/9kPC1aqeoMrIyMNCqh39PBVeMJ5TAkCDRlulo73BDEY
```

The attack: from anywhere, request a reset for the platform admin's address with a forged `Host`. The admin receives a genuine email from the real sender — correct subject, correct copy, correct expiry — and clicks. The 43-character token lands in the attacker's web log. The attacker opens the real `/admin/reset/<token>` within 30 minutes and sets a new admin password. From there, N36 and C3: enter any tenant as owner, or delete it.

The same shape works for every tenant owner (`/{slug}/forgot`) and for every consignor magic link (`/{slug}/consignor/enter`), which is a 30-minute session as that vendor.

Behind a reverse proxy that is configured to overwrite `Host` and strip `X-Forwarded-Host`, this is mitigated — but the repo does not document that requirement, `deploy-demo.sh` does not configure it, and `php -S` (which the HTTP suite and the README both use) does not do it.

Fix: add a required `STALL_PUBLIC_ORIGIN` (or per-tenant canonical host) and build every outbound link from it. Refuse to start if it is unset in non-dev. Have `requestOrigin()` compare the request `Host` to the allow-list and log a warning on mismatch. One day, plus a test that posts a forged `Host` and asserts the mailed link does not contain it.

MEDIUM

N33 — `register_snaps` is fully writable by the poster after posting

`027` ends with `GRANT ALL ON TABLE public.register_snaps TO stall_poster`. The table is an audit record of what the browser claimed and what the ledger did about it (`status`, `refusal`, `journal_id`, `journal_no`). It is also an input to `stall_apply_snap_price()`, which refuses a later sale line that disagrees with the snapped price.

Executed as `stall_poster`: `UPDATE register_snaps SET price_minor = 1 WHERE posting_key = …` → `UPDATE 1` on a `posted` row; `DELETE FROM register_snaps WHERE posting_key = …` → `DELETE 1`. The journal survives (append-only held). The explanation of the journal does not.

Compare `job_mail_sent` (029): `SELECT, INSERT` only. That is the right grant for an append-only evidence table. The same applies to `commission_rules`, `vendor_entry_tokens`, `booth_leases`, `markdown_proposals`, `item_floors` and `vendor_entry_requests`, all of which are `GRANT ALL` to the poster and — more surprisingly — **also `GRANT ALL` to `stall_backup`**, which everywhere else in the schema is `SELECT` only. A backup role that can `DELETE` is a backup role that can be used to destroy what it is meant to preserve.

Fix: `REVOKE UPDATE, DELETE, TRUNCATE ON register_snaps FROM stall_poster, stall_backup;` the sync function already runs the status updates and can be made `SECURITY DEFINER` like the override functions. Audit the seven new tables' grants against the pattern in `029`. Half a day.

N34 — the consignor portal runs as the poster role; `stall_vendor` is unused

`025_vendor_role.sql` creates `stall_vendor` (`NOINHERIT NOLOGIN`). `026_vendor_portal.sql` creates five `security_barrier` views (`stall_vendor_items`, `_sales`, `_payable`, `_agreement`, `_expiring`) filtered on `current_setting('stall.vendor_id')`, grants `SELECT` on them to `stall_vendor` — and also to `stall_poster`. `stall_vendor_open(hash)` resolves a token to a vendor id and `stall_vendor_prefer()` writes a preference.

`VendorPortal::assume()` and `VendorEntry::assume()` both do `$this->roles->assume($tenant->role)` — the poster role. `grep -rn stall_vendor src/` returns nothing. `pg_auth_members` shows the only member of `stall_vendor` is `stall`.

So the portal's isolation is: "PHP sets the GUC to the vendor id from the session, then reads views that filter on the GUC, as a role that could read every table directly". Executed as `stall_poster`:

```
SELECT set_config('stall.vendor_id', '<vendor A>', false);  SELECT count(*) FROM stall_vendor_items;  → 34
SELECT set_config('stall.vendor_id', '<vendor B>', false);  SELECT count(*) FROM stall_vendor_items;  → 8
```

Nothing in the portal code is wrong today. But the entire reason to create `stall_vendor` was so that a bug in the PHP — a mis-set session key, a missing `assume()`, a future controller that forgets the GUC — cannot leak another consignor's figures, because the role it runs as cannot see them. As built, the role that can see everything is the one running the public, token-authenticated, unauthenticated-user-facing page. Of all the roles in the system, the consignor portal is the one that most needs the narrow role, because its users are not staff.

There is also no sign-out for the consignor session (`consignor:<slug>` is set on redeem and never cleared), and no idle expiry — the 30-minute window applies to the token, not the session it opens.

Fix: `GRANT stall_vendor TO stall_app` (and the poster role if the code path requires it), `SET ROLE stall_vendor` in `VendorPortal::assume()`, revoke the views from `stall_poster`, and add a `prove_vendor_portal_role.sql` that reads a base table as `stall_vendor` and asserts refusal. Add a sign-out route and an expiry. One day.

N36 — admin and tenant sessions share one cookie; the admin session never expires

`AdminEnterAs::asOwner()` calls `LoginSession::open()` — which regenerates the id and writes the tenant keys (`owner_slug`, `tenant_id`, `actor_role`, `csrf`, …) — and then `AdminSession::restore($preserved)` to put `platform_admin_id`, `_email`, `_version` back into the same session. Both halves live on `PHPSESSID`.

Executed over HTTP, one cookie jar:

```
POST /admin/login                          → 303 /admin/tenants
POST /admin/tenants/mall/enter role=owner  → 303 /mall/dashboard
GET  /mall/dashboard                       → 200  (Harbor Antique Mall)
GET  /admin/tenants                        → 200  (10 enter buttons)
POST /mall/logout                          → 200 (confirm page)   GET /admin/tenants → 200   ← admin survives tenant logout
POST /admin/logout                         → 303 /admin/login     GET /mall/pos      → 200   ← tenant survives admin logout
```

Consequences. A support engineer who enters a tenant and then clicks the tenant's "Sign out" believes they have left; they have not — the admin half is still live on that browser. One who signs out of `/admin` on a shared machine leaves a tenant owner session behind. `LoginSession::open()` also overwrites `csrf`, so the admin's CSRF token changes under them on enter-as; harmless, but a sign that the two models were not designed together.

Separately, `AdminApp::requireAdmin()` writes `platform_last_seen_at` on every request but never reads `platform_signed_in_at` or `platform_last_seen_at`. Tenant sessions expire at 12 h idle / 24 h absolute (`Actor::stale()`). Admin sessions expire when PHP's garbage collector gets to them. The most privileged session in the system is the only one with no lifetime.

Fix: either put the admin portal on its own cookie name (`session_name('stall_admin')` under `/admin`) so the two cannot share state, or make `/admin/logout` destroy the whole session and make tenant logout clear the admin keys too. Add idle/absolute expiry to `requireAdmin()` mirroring `Actor::stale()`, with shorter limits (an hour idle is plenty for an admin). Half a day.

LOW

N37 — commission and tax triggers mutate `items` and `mall` rows mid-insert (design)

`022` resolves the commission rate for a sale line by *writing* the chosen rate into `items.commission_bps` in a `BEFORE INSERT` trigger, letting the existing rate logic read it, and then restoring the prior value in an `AFTER INSERT` trigger via GUCs (`stall.prior_bps`, `stall.prior_tax`). `027` extends the same trick to `mall.tax_bps`, which is a one-row, store-wide table.

I did not race it. `prove_race_snap_a/b` exist and pass, and the `UPDATE … WHERE id = NEW.item_id` takes a row lock that serialises concurrent sale lines on the same item. But the `UPDATE public.mall SET tax_bps = snap_tax` takes a row lock on the single `mall` row for the duration of the sale-line insert, so two registers posting snaps simultaneously serialise on the settings row; and if

the `AFTER` trigger does not fire (a constraint failure between `BEFORE` and `AFTER` rolls the whole statement back, so this is safe in practice — but only because of transactional semantics the code does not comment on). It is a pattern that works today and that will surprise the next person who adds a trigger between `00_rate` and `zz_rate`.

Once N32 is fixed by computing commission inside the sync function, the `items.commission_bps` rewrite is unnecessary; the snap can pass its rate to the rate trigger through a GUC the same way `stall.snap_priced` already is, without touching `items` or `mall`.

L8 — no security response headers

`/admin/login` returns `Set-Cookie: PHPSESSID=…; path=/; HttpOnly; SameSite=Lax` and nothing else. No `X-Frame-Options` / `frame-ancestors`, no `X-Content-Type-Options: nosniff`, no `Content-Security-Policy`, no `Strict-Transport-Security`, no `Referrer-Policy`. The POS pages embed inline event handlers so a strict CSP is a project, but `frame-ancestors 'none'`, `nosniff` and `Referrer-Policy: same-origin` are three lines in `public/index.php` and close clickjacking of the admin wipe form and referrer leakage of reset tokens. An hour.

Carried, unchanged, and still worth doing

- **C2:** the migration machinery is now genuinely forward-only across 36 files with an `AFTER` phase. Delete the three "create a new database" messages and the README sentence; they are no longer true and they will frighten a buyer's engineer.
- **C3/C4:** the wipe is now better controlled than it was, but it is also now a production feature reachable from a browser. A soft-delete (`platform.tenants.deleted_at`, schema renamed to `gone_<hex>`, purged by the worker after 30 days) would close C3 properly and make C4's legacy path deletable.
- **H8:** the admin side shows what a complete audit trail looks like — append-only, guarded, with `outcome`, `target` and `details`. The tenant side should adopt the same table shape and the same event-trigger guard.
- **N22:** `admin_sign_in_failures` has `client_address`; `override_sign_in_failures` does not. Copy the newer design back.
- **N23:** a dead-lettered job still raises no alert. `check:worker` proves the worker works in CI; nothing proves it is running in production.
- **N29:** `job_mail_sent` still has no retention.
- **L7:** no secret scanning. One workflow step.

What is genuinely good

This is the round where the architecture started paying for itself. The N25/N26/N27 fixes were not patches; they were the right structural answer — tenant-local tables, `SECURITY DEFINER` functions with `REVOKE ALL FROM PUBLIC`, and a platform function that derives the tenant from `current_setting('role')` instead of trusting a parameter. `035` fixing `031`'s `current_user` mistake within the same push shows someone tested it as the actual role. And the drop migrations (`032`, `034`) refusing to run until every tenant has the local table is the kind of care that makes zero-downtime upgrades possible later.

The offline register's idempotency is right. `posting_key` is `UNIQUE`, `stall_sync_snap()` returns the existing journal number on replay, the race proofs cover two sessions syncing the same key, and `LedgerPathTest` drives the whole thing over HTTP including the refusal path. C5's lesson — "never post money without a key" — was learned and applied to the new path before I asked. That is the most important property of an offline POS and it is the one most vendors get wrong.

The admin portal is better built than most SaaS admin consoles I have reviewed. Append-only audit with event-trigger guards so even a schema owner cannot quietly drop the trigger; `credentials_version` so a password change invalidates every other session; per-address lockout with a real `inet` column; a wipe function that validates the slug with a regex, checks for outstanding password resets, and refuses every role but `stall_app`. `AdminPortalTest` exercises login, enter-as owner, enter-as cashier, a wrong slug, and a real deletion, end to end.

`check:worker` is a better gate than the CI step it replaced: it runs as `stall_worker`, asserts every claimed job finished, and asserts nothing due was left behind. The commission-rule engine's "two rules are equally specific" guard trigger turns a classic configuration ambiguity into a refused insert instead of a silent coin-flip. `prove` is at 64 files. PHPStan level 10 is still clean across 10,000 new lines.

And the author copied my last two deliverables into `ninja/` in the repo. That is the right instinct: a review nobody can find is a review nobody acts on.

The three things a competitor will say now

1. **"Your offline mode lets the till decide the commission."** The pitch for offline-tolerant POS is "a sale is never lost and never wrong". Right now the second half is not true: a stale tab or a curious cashier can post a sale that pays the consignor 100%. Fix N32 before the first store runs it on a busy Saturday, because the vendor statements for that month will be wrong and you will find out from the vendor.
2. **"Your password reset can be hijacked with one header."** Host-header poisoning of reset links is on every penetration tester's first-hour checklist. It is also the thing that turns the admin portal from an asset into the single point of failure for every tenant, because an admin who clicks one email can enter or delete every store.
3. **"Your own test suite is red."** Six rounds of this review have leaned on `composer check` being green as the reason to trust the proofs. A buyer's engineer will run it on day one. If it is red, every other claim in the README is discounted.

Suggested order of work

1. **N31** — make the two unit tests assume the poster role / match the new selector; fix the four PHPMD violations; find the `FloorCoverageTest` race; add HTTP tests for `letters.php` and `Admin/reset.php` to lift coverage back over 90%. One day. Then add branch protection so `main` cannot go red again.
2. **N32** — compute commission server-side in `stall_sync_snap()`; post collected tax with a variance line; drop the client rate fields from the snap payload. Add `prove_offline_sync_rules.sql`. Two days.
3. **N35** — `STALL_PUBLIC_ORIGIN`, build every link from it, refuse to start without it outside dev, test with a forged `Host`. One day.
4. **N34** — actually `SET ROLE stall_vendor` in the portal; revoke the views from the poster; add a sign-out and an expiry. One day.
5. **N36** — separate admin cookie or symmetric logout; idle/absolute expiry on `requireAdmin()`. Half a day.
6. **N33** — revoke `UPDATE/DELETE` on the seven new tables from poster and backup; make the sync function `SECURITY DEFINER`. Half a day.
7. **L8, L7** — three headers and one workflow step. Two hours.
8. **C2, C3, H8, N22, N23, N29** — the long tail, now mostly about adopting the admin portal's patterns on the tenant side.

Method and limits

I re-provisioned from scratch, ran all twelve gates plus coverage, and executed the probes listed under "What I executed, not read" against the live cluster and a `php -S` instance. I read `019`–`036` in full, `SnapController`, `SnapSync`, `register-outbox.js`, `ticket-commands.js`, `pay-modal.php`, `VendorEntry`, `VendorEntryController`, `VendorPortal`, `AdminApp`, `AdminSession`, `AdminEnterAs`, `AdminTenantWipe`, `AdminTenantController`, `AdminSessionController`, `AdminPasswordController`, `SaleCeiling`, `JobQueue`, `MailboxSeal`, `wipe-tenant.php`, `check-worker.php`, `check.yml` and the three failing tests.

I did not race N37 (read only). I did not run mutation testing (instruction). I did not exercise the markdown queue, booth leases, or vendor letters beyond confirming their proofs pass — those are the areas most likely to hold the next round's findings. The `FloorCoverageTest` flake is reported, not root-caused. The N35 probe used `php -S`; a production reverse proxy may or may not rewrite `Host`, and the repository does not say.

Probe scripts are in `tmp/`: `probe6.sql` (grants and roles), `snap_probe.sql` (N32/N33), `admin_probe.sh` (N36, lockout), `host_probe.sh` (N35), `gates.sh`, `provision6.sh`.